

# Importing necessary libraries

```
In [1]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import plotly.express as px
from plotly.offline import init_notebook_mode, iplot
init_notebook_mode(connected=True)
```

# Loading Dataset

```
In [2]: df = pd.read_csv("auto_close_tickets.csv")
```

# Exploratory Data Analysis(EDA)

```
In [3]: df.head()
```

Out[3]:

|   | Ticket ID | Customer Name  | Customer Email             | Customer Age | Customer Gender | Product Purchased              | Dat Purcl |
|---|-----------|----------------|----------------------------|--------------|-----------------|--------------------------------|-----------|
| 0 | 1         | Marisa Obrien  | carrollallison@example.com | 32           | Other           | GoPro Hero                     | 2021      |
| 1 | 2         | Jessica Rios   | clarkeashley@example.com   | 42           | Female          | LG Smart TV                    | 2021      |
| 2 | 9         | Nicolas Wilson | joshua24@example.com       | 60           | Other           | Fitbit Versa Smartwatch        | 2020      |
| 3 | 10        | William Dawson | clopez@example.com         | 27           | Male            | Dyson Vacuum Cleaner           | 2020      |
| 4 | 13        | Tamara Hahn    | jensenwilliam@example.net  | 27           | Other           | Nintendo Switch Pro Controller | 2021      |

5 rows × 24 columns

```
In [4]: df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 2881 entries, 0 to 2880
Data columns (total 24 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   Ticket ID                            2881 non-null   int64
1   Customer Name                        2881 non-null   object
2   Customer Email                      2881 non-null   object
3   Customer Age                        2881 non-null   int64
4   Customer Gender                     2881 non-null   object
5   Product Purchased                   2881 non-null   object
6   Date of Purchase                    2881 non-null   object
7   Ticket Type                         2881 non-null   object
8   Ticket Subject                      2881 non-null   object
9   Ticket Description                   2881 non-null   object
10  Ticket Status                       2881 non-null   object
11  Resolution                           0 non-null      float64
12  Ticket Priority                      2881 non-null   object
13  Ticket Channel                      2881 non-null   object
14  First Response Time                 2881 non-null   object
15  Time to Resolution                  0 non-null      float64
16  Customer Satisfaction Rating        0 non-null      float64
17  Age Group                          2881 non-null   object
18  Is Resolved                         2881 non-null   bool
19  Cleaned_Description                 2881 non-null   object
20  Response Delay (in hours)           2881 non-null   float64
21  Resolution Time (in hours)          0 non-null      float64
22  Customer_Response_Pending_Days     2881 non-null   float64
23  Auto_Close                         2881 non-null   bool
dtypes: bool(2), float64(6), int64(2), object(14)
memory usage: 500.9+ KB
```

```
In [5]: sunburst_data = df.groupby(['Age Group', 'Ticket Type', 'Ticket Subject']).size(

fig = px.sunburst(
    sunburst_data,
    path=['Age Group', 'Ticket Type', 'Ticket Subject'],
    values='Count',
    title="Sunburst Chart: Ticket Distribution by Age Group, Type, and Subject"
)

fig.update_layout(
    paper_bgcolor="#E6E6FA",
)

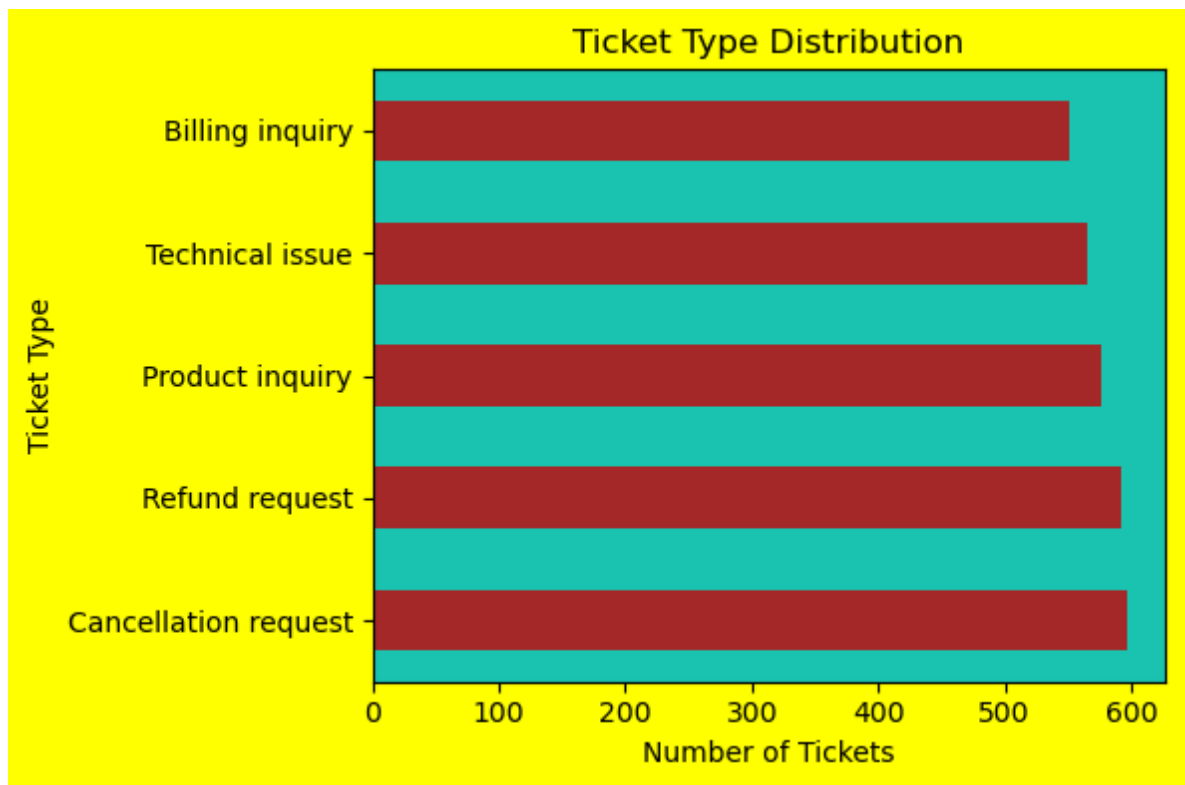
iplot(fig)
```

## Sunburst Chart: Ticket Distribution by Age Group, Type, and

```
In [6]: # Counting the number of each ticket type
ticket_counts = df['Ticket Type'].value_counts()

# Plotting horizontal bar chart
plt.figure(figsize=(6,4),facecolor= 'yellow')
ax= ticket_counts.plot(kind='barh',color="brown")
ax.set_facecolor("#1CC4AF")

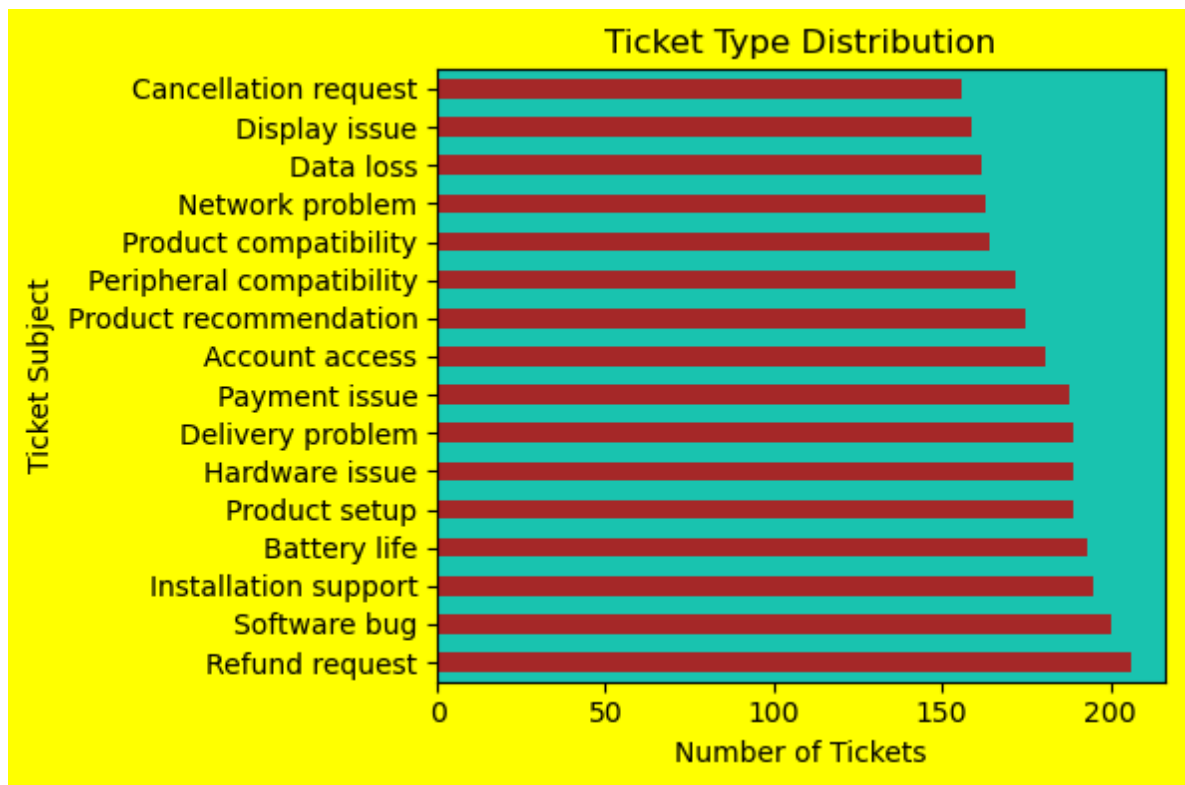
# plot
plt.title('Ticket Type Distribution')
plt.xlabel('Number of Tickets')
plt.ylabel('Ticket Type')
plt.tight_layout()
plt.show()
```



```
In [7]: # Count the number of each ticket subject
ticket_counts = df['Ticket Subject'].value_counts()

# Plotting horizontal bar chart
plt.figure(figsize=(6,4),facecolor= 'yellow')
ax = ticket_counts.plot(kind='barh',color="brown")
ax.set_facecolor("#1CC4AF")

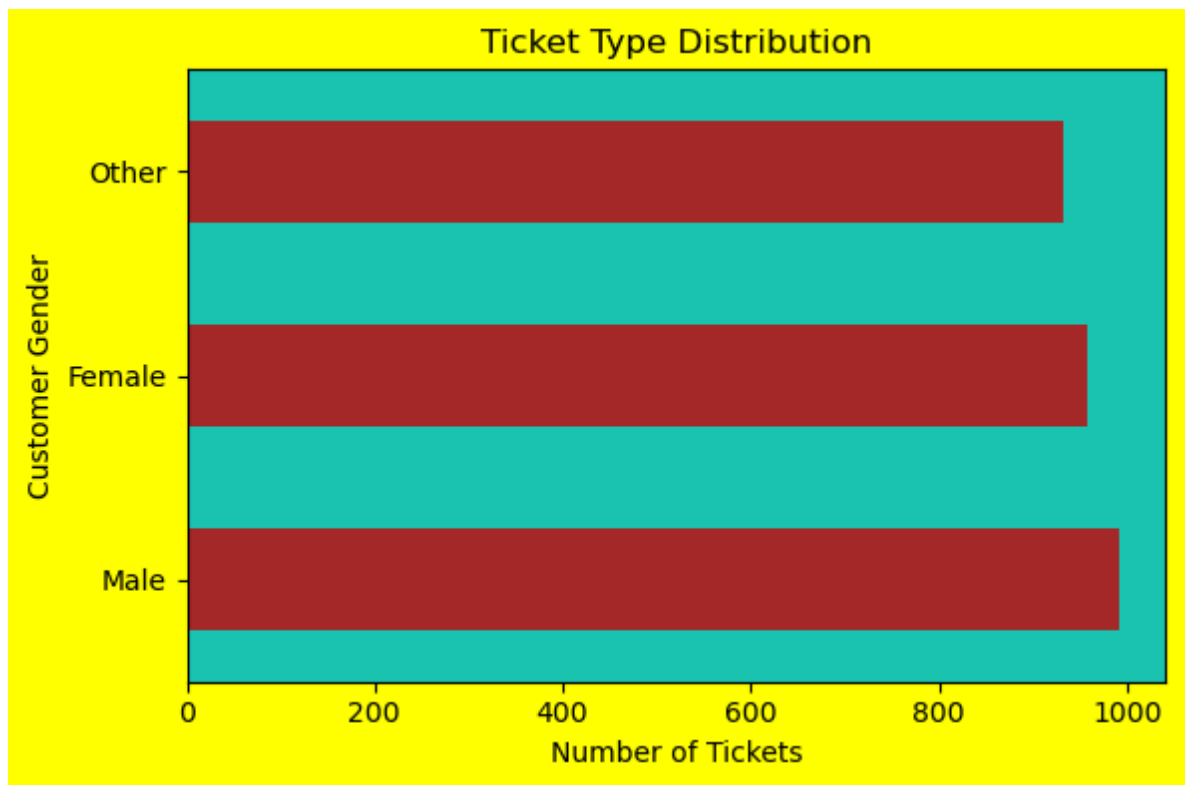
# plot
plt.title('Ticket Type Distribution')
plt.xlabel('Number of Tickets')
plt.ylabel('Ticket Subject')
plt.tight_layout()
plt.show()
```



```
In [8]: # Counting the number of each Gender category
ticket_counts = df['Customer Gender'].value_counts()

# Plotting bar chart
plt.figure(figsize=(6,4),facecolor= 'yellow')
ax = ticket_counts.plot(kind='barh',color="brown")
ax.set_facecolor("#1CC4AF")

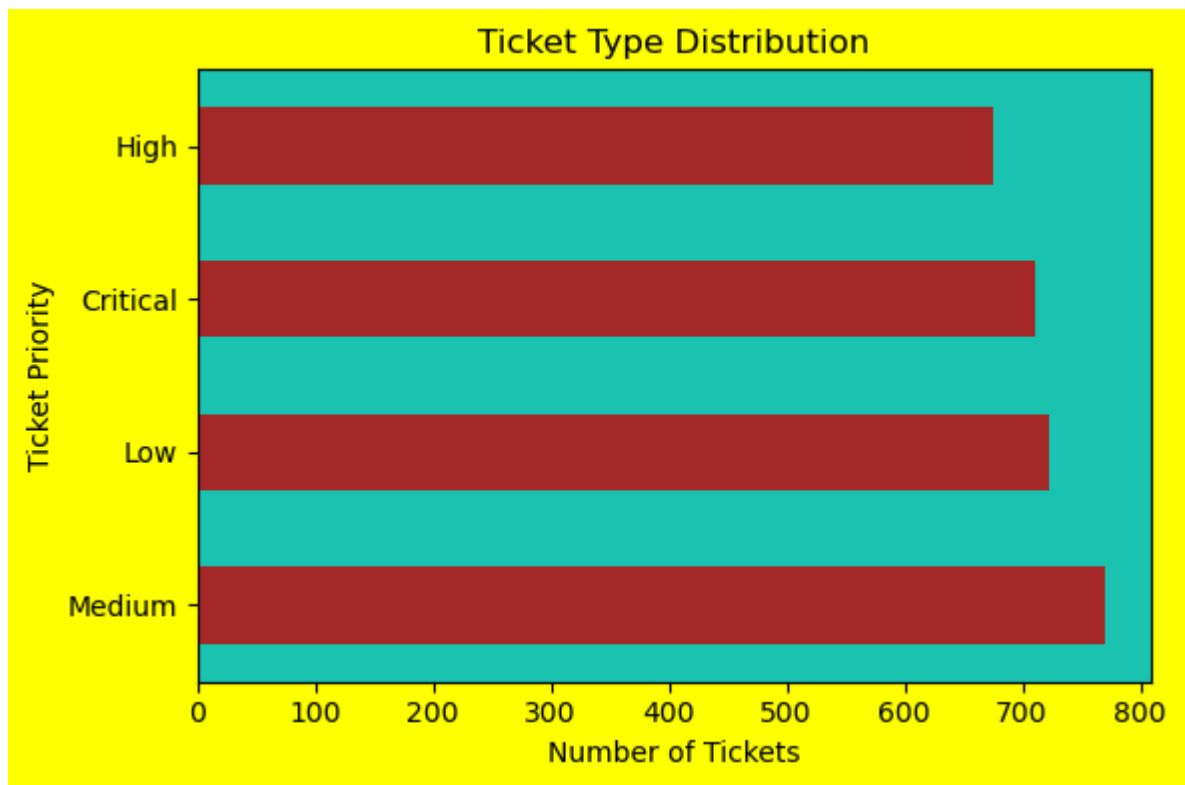
# plot
plt.title('Ticket Type Distribution')
plt.xlabel('Number of Tickets')
plt.ylabel('Customer Gender')
plt.tight_layout()
plt.show()
```



```
In [9]: # Counting the frequency of each ticket priority
ticket_counts = df['Ticket Priority'].value_counts()

# Plotting horizontal bar chart
plt.figure(figsize=(6,4),facecolor= 'yellow')
ax = ticket_counts.plot(kind='barh',color="brown")
ax.set_facecolor("#1CC4AF")

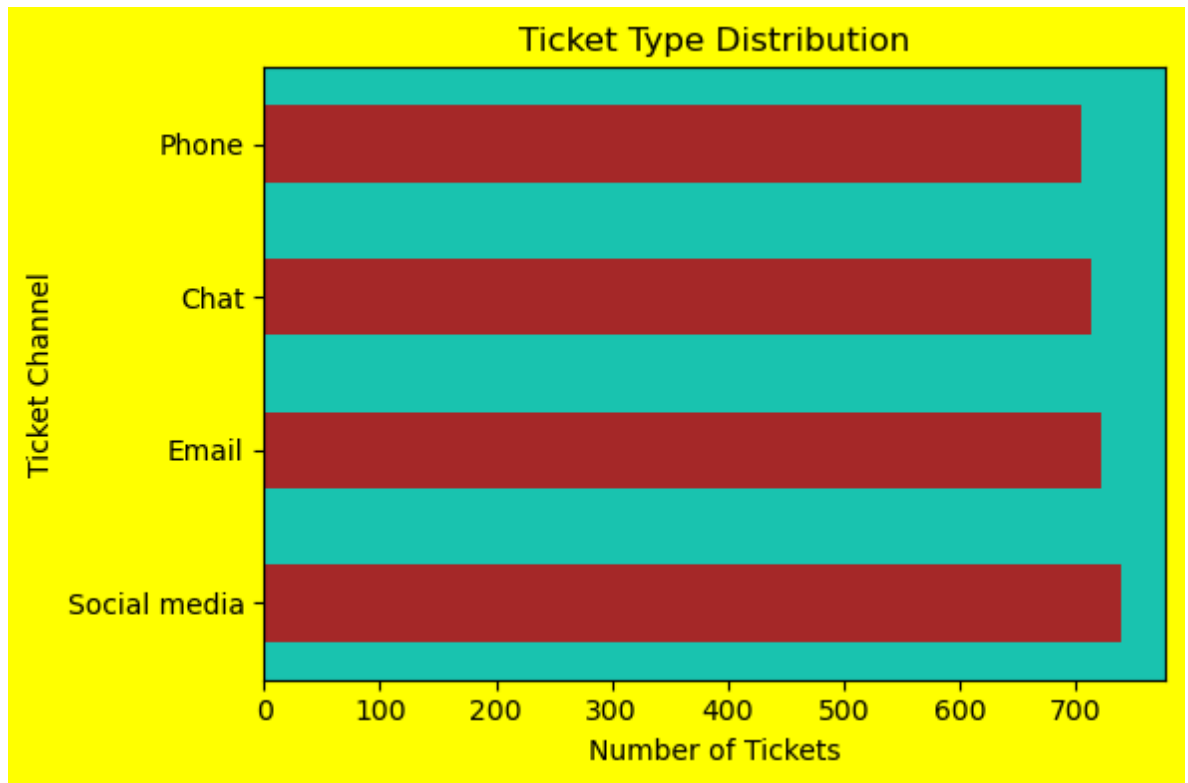
# plot
plt.title('Ticket Type Distribution')
plt.xlabel('Number of Tickets')
plt.ylabel('Ticket Priority')
plt.tight_layout()
plt.show()
```



```
In [10]: # Calculating the distribution of ticket channel
ticket_counts = df['Ticket Channel'].value_counts()

# Plotting horizontal bar chart
plt.figure(figsize=(6,4),facecolor= 'yellow')
ax = ticket_counts.plot(kind='barh',color="brown")
ax.set_facecolor("#1CC4AF")

# plot
plt.title('Ticket Type Distribution')
plt.xlabel('Number of Tickets')
plt.ylabel('Ticket Channel')
plt.tight_layout()
plt.show()
```



## 90-Day Auto-Close Rule for Tickets

### Why we needed this rule:

Sometimes, customers send a ticket and our customer support team replies, but the customer does not respond back. These tickets remain open even though no one is following up. This causes clutter in the system and makes it hard to focus on the tickets that really need attention.

### What we did:

We created an automated rule: If the customer support team has already responded and the customer does not reply within **90 days**, then the system will automatically mark the ticket as resolved and close it.

### How this helps:

- It keeps the support system clean by closing inactive tickets.
- Support agents can focus more on real, open, and urgent issues.
- It saves time and improves the speed of resolving other tickets.
- If the customer still needs help, they can easily raise a new ticket at any time.

### Overall Benefit:

This rule improves the efficiency and organization of customer service, so we can serve customers better and faster.

In [ ]: